AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

ZAAWANSOWANE PROGRAMOWANIE

...Algorytm listy dwukierunkowej z zastosowaniem GitHub...

Autor:
Paweł Pawlik

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

Spis treści

1. Ogólne określenie wymagań **3**

1.1. Założenia programu 3

1.2. Narzędzia i wymagania dla systemu Git 3

2. Analiza problemu **4**

2.1. Jak działa lista dwukierunkowa na stercie 4

2.2. Wykorzystane wzorce projektowe 4

2.3. Podsumowanie operacji na liście 5

3. Projektowanie **6**

4. Implementacja **7**

5. Wnioski **8**

Literatura **9**

Spis rysunków **9**

Spis tabel **10**

Spis listingów **11**

1. Ogólne określenie wymagań

Celem projektu jest napisanie w języku C++ programu obsługującego listę dwukierunkową, która działa na stercie. Oprócz samego kodu ważną częścią zadania jest praca z systemem kontroli wersji Git (wraz z serwisem GitHub) oraz wygenerowanie dokumentacji w programie Doxygen.

1.1. Założenia programu

W projekcie należało spełnić następujące wymagania dotyczące kodu:

- **Podział na pliki:** każda klasa musi znajdować się w osobnym pliku nagłówkowym (`.h`) oraz źródłowym (`.cpp`).
- **Pamięć dynamiczna:** wszystkie elementy listy są tworzone na stercie za pomocą operatora `new` i usuwane operatorem `delete`.
- **Wzorce projektowe:** podczas pisania programu należało wykorzystać wzorzec fabryki (*Factory*) oraz iteratora (*Iterator*).

Lista dwukierunkowa musi udostępniać następujące metody:

- dodawanie elementu na początek, na koniec oraz pod wskazany indeks,
- usuwanie elementu z początku, z końca oraz spod wskazanego indeksu,
- wyświetlanie całej listy od początku oraz w odwrotnej kolejności,
- wyświetlanie następnego i poprzedniego elementu,
- czyszczenie całej listy.

Działanie wszystkich powyższych metod trzeba przetestować w funkcji `main`.

1.2. Narzędzia i wymagania dla systemu Git

Podczas pisania programu kod był na bieżąco zapisywany w repozytorium na platformie GitHub. W ramach projektu należało przetestować podstawowe scenariusze pracy z Gitem:

1. wykonanie co najmniej 5 commitów podczas pisania kodu,
2. cofnięcie się w historii zmian o dwa commity,

3. usunięcie jednego commita,
4. pobranie projektu z GitHuba do innego folderu, dopisanie nowej metody i wysłanie zmian na serwer, a następnie pobranie ich w pierwszym folderze i dalsza praca nad kodem,
5. celowe usunięcie pliku z projektu (tak, aby program przestał się kompilować) i przywrócenie ostatniej działającej wersji z GitHuba.

2. Analiza problemu

Zwykła tablica w C++ ma stały rozmiar i ciężko dodawać do niej nowe elementy w środku lub na początku. Lista dwukierunkowa rozwiązuje ten problem, ponieważ składa się z osobnych węzłów łączonych za pomocą wskaźników, a jej rozmiar może się dynamicznie zmieniać w trakcie działania programu.

2.1. Jak działa lista dwukierunkowa na stercie

Każdy element (węzeł) listy przechowuje trzy rzeczy:

- wartość (w naszym przypadku liczbę całkowitą `int`),
- wskaźnik `next` pokazujący na następny element listy,
- wskaźnik `prev` pokazujący na poprzedni element listy.

Cała lista pamięta wskaźnik na swój pierwszy element (`head`) oraz ostatni (`tail`). Dzięki temu można łatwo przeglądać ją w obie strony – od początku do końca oraz od końca do początku.

Ponieważ lista działa na stercie, każdy nowy węzeł jest tworzony dynamicznie. Bardzo ważne jest tutaj pilnowanie pamięci – przy usuwaniu elementu lub czyszczeniu całej listy trzeba zwolnić pamięć operatorem `delete`, żeby nie dopuścić do wycieków pamięci.

2.2. Wykorzystane wzorce projektowe

Zgodnie z instrukcją w programie zastosowano dwa wzorce projektowe:

- **Factory (Fabryka):** zamiast wpisywać `new` bezpośrednio w metodach listy, tworzeniem nowych węzłów i samej listy zajmuje się osobna klasa `ListFactory`. Dzięki temu kod jest bardziej uporządkowany.

- **Iterator:** osobna klasa, która przechowuje wskaźnik na aktualnie wybrany element listy. Pozwala ona na wygodne przechodzenie po liście krok po kroku oraz wyświetlanie następnego i poprzedniego elementu bez ręcznego operowania na wskaźnikach w funkcji `main`.

2.3. Podsumowanie operacji na liście

W tabeli 2.1 krótko zestawiono najważniejsze operacje wykonywane na liście dwukierunkowej oraz sposób ich działania.

Tab. 2.1. Podstawowe operacje na liście dwukierunkowej

Operacja	**Opis działania w programie**
Dodawanie na początek / koniec	Tworzy nowy węzeł i przepina wskaźniki `head` lub `tail`. Działa bardzo szybko, bez przeglądania całej listy.
Dodawanie / usuwanie pod indeksem	Wymaga przejścia pętlą od początku listy do wybranego miejsca i odpowiedniego przepięcia wskaźników sąsiadów.
Wyświetlanie (przód / tył)	Przechodzi w pętli od `head` (przez `next`) lub od `tail` (przez `prev`) i wypisuje wartości na ekran.
Czyszczenie listy	Usuwa po kolei wszystkie węzły z pamięci za pomocą `delete`, aż lista będzie pusta.

3. Projektowanie

4. Implementacja

5. Wnioski

Spis rysunków

Spis tabel

2.1. Podstawowe operacje na liście dwukierunkowej 5

Spis listingów